Contratto API – ArenaHub

Contratto tra frontend e backend, ricavato dal diagramma ER e dai wireframe. Ogni endpoint indica le schermate che lo usano.

Convenzioni

- **Base URL**: `/api`. Richieste e risposte sono in JSON, tranne il caricamento dell'immagine (multipart).
- **Nomi dei campi**: camelCase nel JSON, snake_case nel database (es. `max_posti` → `postiMax`).
- **Date**: ISO 8601 senza fuso, ora italiana: `"2026-10-16T15:00:00"`.
- **Autenticazione**: dopo il login, ogni richiesta non pubblica manda l'header `Authorization: Bearer <token>`. Il logout si fa nel frontend cancellando il token.
- **Errori**: sempre nel formato `{ "errore": "messaggio leggibile" }`, con il codice HTTP corrispondente.

Livelli di accesso

Livello	Chi
pubblico	chiunque, anche senza login
utente	qualsiasi utente loggato (`ruolo = utente` o `admin`)
capo squadra	utente con `ruolo_squadra = capo_squadra` nella squadra indicata
admin	utente con `ruolo = admin`

Errori comuni

Valgono per tutti gli endpoint e non sono ripetuti sotto.

Codice	Quando	Risposta
400	corpo mancante, campi obbligatori assenti o di tipo sbagliato	`{"errore": "Dati non validi", "campi": ["email"]}`
401	token assente o scaduto su un endpoint non pubblico	`{"errore": "Autenticazione richiesta"}`
403	l'utente non ha il livello di accesso richiesto	`{"errore": "Operazione non consentita"}`
500	errore imprevisto del server	`{"errore": "Errore interno, riprova più tardi"}`

Valori enum

Campo	Valori
`torneo.stato`	`bozza`, `iscrizioni_aperte`, `iscrizioni_chiuse`, `in_corso`, `concluso`, `annullato`
`iscrizione.stato`	`lista_attesa`, `in_attesa`, `confermata`, `ritirata`
`partita.stato`	`programmata`, `in_corso`, `conclusa`
`utente.ruolo`	`admin`, `utente`
`membro_squadra.ruolo_squadra`	`capo_squadra`, `giocatore`, `riserva`

Filtri dei wireframe → `torneo.stato`: Iscrizioni aperte = `iscrizioni_aperte`, In corso = `in_corso`, Conclusi = `concluso`, Cancellati = `annullato`, Bozze = `bozza` (solo admin).

Un'iscrizione occupa un posto quando è `in_attesa` (da confermare) o `confermata`. Le iscrizioni in `lista_attesa` e `ritirata` non occupano posti.

Prima di implementare: aggiunte al diagramma ER

I wireframe usano dati che nel diagramma attuale non ci sono. Senza queste aggiunte alcuni endpoint non si possono implementare.

Aggiunta	Perché	Endpoint coinvolti
`torneo.data_inizio`, `torneo.data_fine` (DATETIME)	date del torneo nelle card, nel dettaglio e nel form di creazione	tornei
`torneo.regolamento` (TEXT)	campo del form di creazione e link «Leggi il regolamento»	tornei
tabella `preferito` (`fk_utente`, `fk_torneo`, PK composta, `data_salvataggio`)	pagina «I miei preferiti» e pulsante Salva	preferiti
`squadra.fk_creatore` (FK → utente)	la relazione «crea» con limite di 10 squadre per utente non ha ancora una colonna	squadre

---

Mappa schermate → endpoint

Schermata del wireframe	Endpoint
Login, Login · errore	`POST /api/auth/login`
Registrazione, Registrazione · errore	`POST /api/auth/registrazione`
Header («Ciao»), Modifica profilo	`GET /api/auth/me`, `PUT /api/utenti/me`
Tornei, I miei tornei, Gestione tornei	`GET /api/tornei`
Dettaglio torneo	`GET /api/tornei/{id}`, `GET /api/squadre/mie`
Iscriviti, Lista d'attesa	`POST /api/tornei/{id}/iscrizioni`
Disiscriviti, conferma disiscrizione	`DELETE /api/iscrizioni/{id}`
I miei preferiti, pulsante Salva	`GET`, `POST`, `DELETE /api/preferiti`
Creazione torneo	`GET /api/tipologie`, `GET /api/giochi`, `GET /api/piattaforme`, `POST /api/tornei`, `POST /api/tornei/{id}/immagine`
Gestisci torneo	`PUT /api/tornei/{id}`, `PATCH /api/tornei/{id}/stato`, `GET /api/tornei/{id}/iscrizioni`, `PATCH /api/iscrizioni/{id}`, `DELETE /api/iscrizioni/{id}`, `POST /api/tornei/{id}/tabellone`
Tabellone	`GET /api/tornei/{id}/partite`, `PATCH /api/partite/{id}/risultato`

Autenticazione

POST /api/auth/registrazione

Crea un nuovo account con ruolo `utente`. Accesso: pubblico. Schermate: Registrazione, Registrazione · errore.

Parametro	Dove	Tipo	Obbligatorio
email	body	stringa (email valida)	sì
nickname	body	stringa (3–20 caratteri)	sì
nome	body	stringa	sì
cognome	body	stringa	sì
password	body	stringa (min 8 caratteri)	sì

Il campo «Conferma password» del form è controllato solo nel frontend e non viene inviato.

Risposta 201

```
{ "id": 10, "nickname": "marco99", "nome": "Marco", "cognome": "Rossi",
  "email": "marco@mail.it", "ruolo": "utente" }
```

Errori

Codice	Quando	Risposta
409	email già registrata	`{"errore": "Email già registrata", "campo": "email"}`
409	nickname già usato	`{"errore": "Nickname già in uso", "campo": "nickname"}`

POST /api/auth/login

Verifica le credenziali e restituisce il token. Accesso: pubblico. Schermate: Login, Login · errore.

Parametro	Dove	Tipo	Obbligatorio
email	body	stringa	sì
password	body	stringa	sì

Risposta 200

```
{ "token": "eyJhbGciOi...",
  "utente": { "id": 10, "nickname": "marco99", "ruolo": "utente" } }
```

Il frontend usa `utente.ruolo` per decidere se mostrare l'area admin.

Errori

Codice	Quando	Risposta
401	email o password errate	`{"errore": "Email o password non corretti"}`

GET /api/auth/me

Dati dell'utente loggato: alimentano il pulsante «Ciao» e precompilano Modifica profilo. Accesso: utente. Schermate: header di tutte le pagine da loggato, Modifica profilo.

Nessun parametro.

Risposta 200

```
{ "id": 10, "nickname": "marco99", "nome": "Marco", "cognome": "Rossi",
  "email": "marco@mail.it", "ruolo": "utente" }
```

Utenti

PUT /api/utenti/me

Modifica i dati del proprio profilo. Si inviano solo i campi cambiati. Accesso: utente. Schermata: Modifica profilo.

Parametro	Dove	Tipo	Obbligatorio
nickname	body	stringa (3–20 caratteri)	no
email	body	stringa (email valida)	no
nome	body	stringa	no
cognome	body	stringa	no
password	body	stringa (min 8 caratteri)	no; se assente la password non cambia

Risposta 200

```
{ "id": 10, "nickname": "marco_r", "nome": "Marco", "cognome": "Rossi",
  "email": "marco@mail.it", "ruolo": "utente" }
```

Errori

Codice	Quando	Risposta
409	email già usata da un altro utente	`{"errore": "Email già registrata", "campo": "email"}`
409	nickname già usato da un altro utente	`{"errore": "Nickname già in uso", "campo": "nickname"}`

Cataloghi

Elenchi usati dalle tendine del form di creazione torneo. I dati vengono inseriti direttamente nel database.

GET /api/tipologie

Elenco delle tipologie di gioco. Accesso: pubblico. Schermata: Creazione torneo (tendina «Categoria gioco»).

Nessun parametro.

Risposta 200

```
[ { "id": 1, "nome": "FPS" }, { "id": 2, "nome": "Sportivo" }, { "id": 3, "nome": "Picchiaduro" } ]
```

GET /api/giochi

Elenco dei giochi, filtrabile per tipologia. Accesso: pubblico. Schermata: Creazione torneo (tendina «Gioco», che si aggiorna quando cambia la categoria).

Parametro	Dove	Tipo	Obbligatorio
tipologia	query	intero (`id_tipologia`)	no

Risposta 200

```
[ { "id": 2, "nome": "Valorant", "descrizione": "Sparatutto tattico 5 contro 5",
    "tipologia": { "id": 1, "nome": "FPS" } } ]
```

GET /api/piattaforme

Elenco delle piattaforme. Accesso: pubblico. Schermata: Creazione torneo (tendina «Piattaforme», selezione multipla).

Nessun parametro.

Risposta 200

```
[ { "id": 1, "nome": "PC" }, { "id": 2, "nome": "PS5" }, { "id": 3, "nome": "Xbox" }, { "id": 4, "nome": "Switch" } ]
```

Tornei

GET /api/tornei

Elenco paginato dei tornei, con i dati delle card. Accesso: pubblico (con `iscritto=me` serve il login). Schermate: Tornei, I miei tornei, Gestione tornei (admin).

Parametro	Dove	Tipo	Obbligatorio
stato	query	stringa (enum `torneo.stato`)	no; senza filtro restituisce tutti tranne le bozze
iscritto	query	`me`	no; solo i tornei con un'iscrizione dell'utente non ritirata
pagina	query	intero (default 1, 10 tornei per pagina)	no

I tornei in `bozza` sono restituiti solo all'admin.

Se la richiesta contiene un token valido, ogni torneo riporta anche lo stato dell'utente:

- `mioStato`: stato della sua iscrizione, `null` se non è iscritto;
- `salvato`: `true` se il torneo è tra i suoi preferiti.

Senza token entrambi i campi valgono `null`. Il frontend li usa per scegliere il pulsante della card: Iscriviti, Disiscriviti, In attesa, Lista d'attesa.

Risposta 200

```
{ "pagina": 1, "pagineTotali": 2,
  "tornei": [
    { "id": 3, "nome": "Valorant Cup", "urlImmagine": "https://…/valorant-cup.png",
      "gioco": "Valorant", "piattaforme": ["PC"], "giocatoriPerSquadra": 5,
      "dataInizio": "2026-10-16T15:00:00", "postiMax": 16, "iscritti": 7,
      "stato": "iscrizioni_aperte", "mioStato": "confermata", "salvato": true },
    { "id": 5, "nome": "Sfida di Natale", "urlImmagine": "https://…/natale.png",
      "gioco": "Mario Kart", "piattaforme": ["Switch"], "giocatoriPerSquadra": 1,
      "dataInizio": "2026-12-18T16:00:00", "postiMax": 16, "iscritti": 16,
      "stato": "iscrizioni_aperte", "mioStato": null, "salvato": false } ] }
```

Una lista vuota restituisce `200` con `"tornei": []`: il frontend mostra «Nessun torneo».

Errori

Codice	Quando	Risposta
400	`stato` non è un valore ammesso	`{"errore": "Stato non valido"}`
401	`iscritto=me` senza login	`{"errore": "Autenticazione richiesta"}`

GET /api/tornei/{id}

Dettaglio completo di un torneo. Accesso: pubblico (i tornei in `bozza` solo admin). Schermate: Dettaglio torneo (visitatore, iscriviti, iscritto, salva, conferma disiscrizione), Gestisci torneo.

Parametro	Dove	Tipo	Obbligatorio
id	path	intero	sì

`miaIscrizione` è `null` se l'utente non è loggato o non è iscritto. Il frontend la usa per scegliere tra «Accedi per iscriverti», «Iscriviti» e «Disiscriviti», e per mostrare la squadra iscritta.

Risposta 200

```
{ "id": 3, "nome": "Valorant Cup",
  "descrizione": "Torneo 5 contro 5 aperto a tutte le squadre.",
  "regolamento": "Formato a eliminazione diretta, partite al meglio di una mappa…",
  "urlImmagine": "https://…/valorant-cup.png",
  "gioco": { "id": 2, "nome": "Valorant", "tipologia": "FPS" },
  "piattaforme": [ { "id": 1, "nome": "PC" } ],
  "giocatoriPerSquadra": 5, "postiMax": 16, "iscritti": 7, "premio": "200 €",
  "aperturaIscrizioni": "2026-10-01T00:00:00", "chiusuraIscrizioni": "2026-10-15T23:59:00",
  "dataInizio": "2026-10-16T15:00:00", "dataFine": "2026-10-16T20:00:00",
  "stato": "iscrizioni_aperte",
  "organizzatore": { "id": 1, "nickname": "admin" },
  "miaIscrizione": { "id": 41, "stato": "confermata", "squadra": { "id": 5, "nome": "Team Alpha" },
                     "posizioneAttesa": null },
  "salvato": true }
```

Errori

Codice	Quando	Risposta
404	id inesistente, o torneo in bozza richiesto da un non admin	`{"errore": "Torneo non trovato"}`

POST /api/tornei

Crea un torneo in stato `bozza`; l'admin che lo crea diventa organizzatore. Accesso: admin. Schermata: Creazione torneo.

Parametro	Dove	Tipo	Obbligatorio
nome	body	stringa	sì
idGioco	body	intero	sì
idPiattaforme	body	array di interi (almeno 1)	sì
premio	body	stringa (es. "200 €")	no
dataInizio	body	data-ora ISO	sì
dataFine	body	data-ora ISO	no
aperturaIscrizioni	body	data-ora ISO	sì
chiusuraIscrizioni	body	data-ora ISO	sì
giocatoriPerSquadra	body	intero (≥ 1; 1 = individuale)	sì
postiMax	body	intero (≥ 2)	sì
descrizione	body	stringa	no
regolamento	body	stringa	no

L'immagine si carica dopo, con `POST /api/tornei/{id}/immagine`, usando l'`id` restituito.

Risposta 201


```
{ "id": 6, "nome": "Torneo d'inverno", "stato": "bozza",
  "gioco": { "id": 7, "nome": "Calcio virtuale", "tipologia": "Sportivo" },
  "piattaforme": [ { "id": 2, "nome": "PS5" } ],
  "giocatoriPerSquadra": 1, "postiMax": 32, "premio": "100 €",
  "aperturaIscrizioni": "2026-11-20T00:00:00", "chiusuraIscrizioni": "2026-12-10T23:59:00",
  "dataInizio": "2026-12-12T15:00:00", "dataFine": "2026-12-12T20:00:00",
  "urlImmagine": null }
```


Errori

Codice	Quando	Risposta
404	gioco o piattaforma inesistente	`{"errore": "Gioco non trovato"}`
422	date incoerenti (apertura dopo chiusura, chiusura dopo inizio, fine prima di inizio)	`{"errore": "Le iscrizioni devono chiudersi prima dell'inizio del torneo"}`

POST /api/tornei/{id}/immagine

Carica o sostituisce l'immagine del torneo e ne salva l'indirizzo in `url_image`. Accesso: admin. Schermate: Creazione torneo, Gestisci torneo (Modifica dati).

Parametro	Dove	Tipo	Obbligatorio
id	path	intero	sì
file	body (multipart/form-data)	immagine jpg o png, max 2 MB	sì

Risposta 200


```
{ "id": 6, "urlImmagine": "https://…/tornei/6.png" }
```


Errori

Codice	Quando	Risposta
404	torneo inesistente	`{"errore": "Torneo non trovato"}`
413	file oltre 2 MB	`{"errore": "Immagine troppo grande (max 2 MB)"}`
415	formato diverso da jpg o png	`{"errore": "Formato non supportato: usa jpg o png"}`

PUT /api/tornei/{id}

Modifica i dati del torneo. Si inviano solo i campi cambiati; i nomi sono quelli di `POST /api/tornei`. Accesso: admin. Schermata: Gestisci torneo (Modifica dati).

Parametro	Dove	Tipo	Obbligatorio
id	path	intero	sì
campi di POST /api/tornei	body	come sopra	no

Consentito solo negli stati `bozza`, `iscrizioni_aperte` e `iscrizioni_chiuse`. Gioco e giocatori per squadra non si possono cambiare se ci sono già iscrizioni.

Risposta 200 – il torneo aggiornato, nello stesso formato di `GET /api/tornei/{id}`.

Errori

Codice	Quando	Risposta
404	torneo inesistente	`{"errore": "Torneo non trovato"}`
409	torneo in corso, concluso o annullato	`{"errore": "Il torneo non è più modificabile"}`
409	cambio di gioco o di giocatori per squadra con iscrizioni presenti	`{"errore": "Ci sono già iscrizioni: gioco e formato non sono modificabili"}`
422	`postiMax` inferiore ai posti già occupati	`{"errore": "Ci sono già 7 iscrizioni"}`
422	date incoerenti	`{"errore": "Le iscrizioni devono chiudersi prima dell'inizio del torneo"}`

PATCH /api/tornei/{id}/stato

Cambia lo stato del torneo. Accesso: admin. Schermate: Gestisci torneo (tendina Stato, pulsante «Annulla torneo»).

Transizioni ammesse: `bozza → iscrizioni_aperte → iscrizioni_chiuse → in_corso → concluso`. Si può passare ad `annullato` da qualsiasi stato tranne `concluso`.

Parametro	Dove	Tipo	Obbligatorio
id	path	intero	sì
stato	body	stringa (enum `torneo.stato`)	sì

Risposta 200

```
{ "id": 3, "stato": "iscrizioni_chiuse" }
```

Errori

Codice	Quando	Risposta
404	torneo inesistente	`{"errore": "Torneo non trovato"}`
422	transizione non ammessa	`{"errore": "Impossibile passare da concluso a in_corso"}`
422	passaggio a `in_corso` senza tabellone generato	`{"errore": "Genera il tabellone prima di avviare il torneo"}`

Iscrizioni

POST /api/tornei/{id}/iscrizioni

Iscrive al torneo l'utente loggato (tornei 1 vs 1) o una sua squadra (tornei a squadre). Accesso: utente; per le squadre serve essere capo squadra. Schermate: Dettaglio torneo (Iscriviti), Iscrizione completata, Lista d'attesa.

Parametro	Dove	Tipo	Obbligatorio
id	path	intero	sì
idSquadra	body	intero	solo se `giocatoriPerSquadra > 1`

Lo stato iniziale dipende dai posti:

- se ci sono posti liberi, l'iscrizione nasce `in_attesa` e l'admin la conferma da Gestisci torneo;
- se il torneo è pieno, nasce `lista_attesa` e la risposta indica la posizione in coda.

Risposta 201 – posto disponibile

```
{ "id": 41, "idTorneo": 3, "stato": "in_attesa",
  "squadra": { "id": 5, "nome": "Team Alpha" }, "utente": null,
  "dataIscrizione": "2026-10-12T11:00:00", "posizioneAttesa": null }
```

Risposta 201 – torneo pieno

```
{ "id": 52, "idTorneo": 5, "stato": "lista_attesa",
  "squadra": null, "utente": { "id": 10, "nickname": "marco99" },
  "dataIscrizione": "2026-10-12T11:05:00", "posizioneAttesa": 2 }
```

Errori

Codice	Quando	Risposta
404	torneo o squadra inesistente	`{"errore": "Torneo non trovato"}`
403	l'utente non è capo della squadra indicata	`{"errore": "Solo il capo squadra può iscrivere la squadra"}`
409	utente o squadra già iscritti (iscrizione non ritirata)	`{"errore": "Già iscritto a questo torneo"}`
422	torneo non in `iscrizioni_aperte` o fuori dalle date di iscrizione	`{"errore": "Le iscrizioni non sono aperte"}`
422	squadra mancante in un torneo a squadre, o indicata in un torneo 1 vs 1	`{"errore": "Questo torneo è a squadre: scegli la squadra da iscrivere"}`
422	squadra con meno membri di `giocatoriPerSquadra`	`{"errore": "La squadra deve avere almeno 5 membri"}`

GET /api/tornei/{id}/iscrizioni

Elenco degli iscritti a un torneo. Accesso: admin. Schermata: Gestisci torneo (tabella Iscritti).

Parametro	Dove	Tipo	Obbligatorio
id	path	intero	sì
stato	query	stringa (enum `iscrizione.stato`)	no; senza filtro esclude le ritirate
pagina	query	intero (default 1, 20 per pagina)	no

Risposta 200

```
{ "pagina": 1, "pagineTotali": 2,
  "iscrizioni": [
    { "id": 41, "nome": "Team Alpha", "tipo": "squadra", "dataIscrizione": "2026-10-02T10:15:00",
      "stato": "confermata", "posizioneAttesa": null },
    { "id": 44, "nome": "Night Owls", "tipo": "squadra", "dataIscrizione": "2026-10-03T18:40:00",
      "stato": "in_attesa", "posizioneAttesa": null },
    { "id": 49, "nome": "Blue Sharks", "tipo": "squadra", "dataIscrizione": "2026-10-09T09:00:00",
      "stato": "lista_attesa", "posizioneAttesa": 1 } ] }
```

`nome` è il nome della squadra, oppure il nickname dell'utente nei tornei 1 vs 1.

Errori

Codice	Quando	Risposta
404	torneo inesistente	`{"errore": "Torneo non trovato"}`

PATCH /api/iscrizioni/{id}

Conferma un'iscrizione in attesa. Accesso: admin. Schermata: Gestisci torneo (pulsante Conferma).

Parametro	Dove	Tipo	Obbligatorio
id	path	intero (`id_iscrizione`)	sì
stato	body	`confermata`	sì

Risposta 200

```
{ "id": 44, "idTorneo": 3, "stato": "confermata" }
```

Errori

Codice	Quando	Risposta
404	iscrizione inesistente	`{"errore": "Iscrizione non trovata"}`
422	iscrizione non `in_attesa` (ritirata o in lista d'attesa)	`{"errore": "Si possono confermare solo le iscrizioni in attesa"}`

DELETE /api/iscrizioni/{id}

Ritira un'iscrizione: lo stato diventa `ritirata` e il record resta nel database. Accesso: il titolare dell'iscrizione (l'utente stesso, o il capo squadra per le squadre) oppure admin. Schermate: Conferma disiscrizione, Disiscrizione completata, I miei tornei, Gestisci torneo (Rimuovi).

Parametro	Dove	Tipo	Obbligatorio
id	path	intero (`id_iscrizione`)	sì

Se si libera un posto, la prima iscrizione in `lista_attesa` (la più vecchia per `data_iscrizione`) passa a `in_attesa`. La risposta indica quale.

Risposta 200

```
{ "id": 41, "idTorneo": 3, "stato": "ritirata",
  "promossa": { "id": 49, "nome": "Blue Sharks", "stato": "in_attesa" } }
```

`promossa` è `null` se nessuno era in lista d'attesa.

Errori

Codice	Quando	Risposta
403	l'utente non è titolare dell'iscrizione	`{"errore": "Non puoi ritirare questa iscrizione"}`
404	iscrizione inesistente	`{"errore": "Iscrizione non trovata"}`
409	iscrizione già ritirata	`{"errore": "Iscrizione già ritirata"}`
409	torneo già iniziato (vale solo per gli utenti, non per l'admin)	`{"errore": "Il torneo è già iniziato"}`

Preferiti

> Richiede la tabella `preferito` (vedi «Aggiunte al diagramma ER»).

GET /api/preferiti

Tornei salvati dall'utente, nello stesso formato delle card di `GET /api/tornei`. Accesso: utente. Schermata: I miei preferiti.

Parametro	Dove	Tipo	Obbligatorio
stato	query	stringa (enum `torneo.stato`)	no
pagina	query	intero (default 1)	no

Risposta 200

```
{ "pagina": 1, "pagineTotali": 1,
  "tornei": [
    { "id": 3, "nome": "Valorant Cup", "urlImmagine": "https://…/valorant-cup.png",
      "gioco": "Valorant", "piattaforme": ["PC"], "giocatoriPerSquadra": 5,
      "dataInizio": "2026-10-16T15:00:00", "postiMax": 16, "iscritti": 7,
      "stato": "iscrizioni_aperte", "mioStato": "confermata", "salvato": true } ] }
```

POST /api/preferiti

Salva un torneo tra i preferiti. Accesso: utente. Schermate: Dettaglio torneo (pulsante Salva), Torneo salvato.

Parametro	Dove	Tipo	Obbligatorio
idTorneo	body	intero	sì

Risposta 201

```
{ "idTorneo": 3, "dataSalvataggio": "2026-10-12T17:00:00" }
```

Errori

Codice	Quando	Risposta
404	torneo inesistente	`{"errore": "Torneo non trovato"}`
409	torneo già tra i preferiti	`{"errore": "Torneo già salvato"}`

DELETE /api/preferiti/{idTorneo}

Toglie un torneo dai preferiti. Accesso: utente. Schermate: I miei preferiti (Rimuovi), Dettaglio torneo (secondo click su Salva).

Parametro	Dove	Tipo	Obbligatorio
idTorneo	path	intero	sì

Risposta 204 – nessun contenuto.

Errori

Codice	Quando	Risposta
404	il torneo non è tra i preferiti	`{"errore": "Torneo non presente nei preferiti"}`

Partite e tabellone

POST /api/tornei/{id}/tabellone

Genera le partite del primo turno accoppiando a caso le iscrizioni confermate. Accesso: admin. Schermata: Gestisci torneo (Genera tabellone).

Il torneo deve essere in `iscrizioni_chiuse`. Se il numero di partecipanti non è una potenza di 2, alcuni passano direttamente al turno successivo.

Parametro	Dove	Tipo	Obbligatorio
id	path	intero	sì
dataPartita	body	data-ora ISO (orario del primo turno)	no; default `dataInizio` del torneo

Risposta 201

```
{ "idTorneo": 3, "round": 1, "partiteCreate": 4,
  "partite": [ { "id": 21, "iscrizioneA": 41, "iscrizioneB": 44 },
               { "id": 22, "iscrizioneA": 45, "iscrizioneB": 49 } ] }
```

Errori

Codice	Quando	Risposta
404	torneo inesistente	`{"errore": "Torneo non trovato"}`
409	tabellone già generato	`{"errore": "Il tabellone esiste già"}`
422	torneo non in `iscrizioni_chiuse`	`{"errore": "Chiudi le iscrizioni prima di generare il tabellone"}`
422	meno di 2 iscrizioni confermate	`{"errore": "Servono almeno 2 partecipanti confermati"}`

GET /api/tornei/{id}/partite

Partite del torneo raggruppate per turno, per disegnare il tabellone. Accesso: pubblico. Schermata: Tabellone.

Parametro	Dove	Tipo	Obbligatorio
id	path	intero	sì

I partecipanti sono `null` finché non sono noti («da definire» nel wireframe). Punteggi e vincitore sono `null` finché la partita non è

conclusa.

Risposta 200


```
{ "idTorneo": 3, "turni": [
  { "round": 1, "partite": [
    { "id": 21, "stato": "conclusa", "dataPartita": "2026-10-16T15:00:00",
      "partecipanteA": { "idIscrizione": 41, "nome": "Team Alpha" }, "punteggioA": 13,
      "partecipanteB": { "idIscrizione": 44, "nome": "Night Owls" }, "punteggioB": 7,
      "vincitore": 41 },
    { "id": 24, "stato": "programmata", "dataPartita": "2026-10-16T16:30:00",
      "partecipanteA": { "idIscrizione": 47, "nome": "Neon Cats" }, "punteggioA": null,
      "partecipanteB": { "idIscrizione": 48, "nome": "Storm FC" }, "punteggioB": null,
      "vincitore": null } ] },
  { "round": 2, "partite": [
    { "id": 25, "stato": "programmata", "dataPartita": "2026-10-16T18:00:00",
      "partecipanteA": { "idIscrizione": 41, "nome": "Team Alpha" }, "punteggioA": null,
      "partecipanteB": null, "punteggioB": null, "vincitore": null } ] } ] }
```


Errori

Codice	Quando	Risposta
404	torneo inesistente	`{"errore": "Torneo non trovato"}`

PATCH /api/partite/{id}/risultato

Registra il punteggio di una partita. Il server calcola il vincitore, segna la partita come `conclusa` e porta il vincitore nella partita del turno successivo. Dopo la finale, il torneo passa a `concluso`. Accesso: admin. Schermata: Tabellone (Inserisci risultato).

Parametro	Dove	Tipo	Obbligatorio
id	path	intero	sì
punteggioA	body	intero (≥ 0)	sì
punteggioB	body	intero (≥ 0)	sì

Risposta 200


```
{ "id": 21, "stato": "conclusa", "punteggioA": 13, "punteggioB": 7,
  "vincitore": { "idIscrizione": 41, "nome": "Team Alpha" },
  "prossimaPartita": 25 }
```


`prossimaPartita` è `null` per la finale.

Errori

Codice	Quando	Risposta
404	partita inesistente	`{"errore": "Partita non trovata"}`
409	torneo non `in_corso`	`{"errore": "Il torneo non è in corso"}`
422	manca uno dei due partecipanti	`{"errore": "La partita non ha ancora due partecipanti"}`
422	punteggi pari (non ammessi a eliminazione diretta)	`{"errore": "Il pareggio non è ammesso"}`

Squadre

Il wireframe usa le squadre solo nella tendina «Squadra da iscrivere» del dettaglio torneo. Le pagine per creare una squadra e gestirne i membri non sono ancora disegnate; gli endpoint sotto le rendono possibili.

GET /api/squadre/mie

Squadre di cui l'utente fa parte, con il suo ruolo. Accesso: utente. Schermata: Dettaglio torneo (tendina «Squadra da iscrivere»: il frontend mostra solo quelle con `mioRuolo = capo_squadra`).

Nessun parametro.

Risposta 200

```
[ { "id": 5, "nome": "Team Alpha", "mioRuolo": "capo_squadra", "membri": 6 },
  { "id": 8, "nome": "Night Owls", "mioRuolo": "giocatore", "membri": 5 } ]
```

POST /api/squadre

Crea una squadra; chi la crea ne diventa capo squadra. Ogni utente può creare al massimo 10 squadre. Accesso: utente. Schermata: da disegnare.

Parametro	Dove	Tipo	Obbligatorio
nome	body	stringa	sì

Risposta 201

```
{ "id": 9, "nome": "Red Foxes",
  "membri": [ { "idUtente": 10, "nickname": "marco99", "ruoloSquadra": "capo_squadra" } ] }
```

Errori

Codice	Quando	Risposta
409	nome già usato	`{"errore": "Nome squadra già in uso"}`
422	l'utente ha già creato 10 squadre	`{"errore": "Hai raggiunto il limite di 10 squadre"}`

POST /api/squadre/{id}/membri

Aggiunge un utente alla squadra cercandolo per nickname. Una squadra ha al massimo 28 membri. Accesso: capo squadra. Schermata: da disegnare.

Parametro	Dove	Tipo	Obbligatorio
id	path	intero	sì
nickname	body	stringa	sì
ruoloSquadra	body	`giocatore` o `riserva`	no (default `giocatore`)

Risposta 201

```
{ "idSquadra": 5, "idUtente": 13, "nickname": "dario_k", "ruoloSquadra": "giocatore" }
```

Errori

Codice	Quando	Risposta
404	squadra o nickname inesistente	`{"errore": "Utente non trovato"}`
409	utente già nella squadra	`{"errore": "L'utente fa già parte della squadra"}`
422	squadra già a 28 membri	`{"errore": "La squadra ha raggiunto il numero massimo di membri"}`

DELETE /api/squadre/{id}/membri/{idUtente}

Rimuove un membro, oppure permette a un membro di uscire. Il capo squadra può uscire solo quando è l'ultimo membro: in quel caso la squadra viene eliminata. Accesso: capo squadra, o l'utente stesso. Schermata: da disegnare.

Parametro	Dove	Tipo	Obbligatorio
id	path	intero	sì
idUtente	path	intero	sì

Risposta 204 – nessun contenuto.

Errori

Codice	Quando	Risposta
404	membro inesistente	`{"errore": "Membro non trovato"}`
409	il capo squadra tenta di uscire con altri membri presenti	`{"errore": "Il capo squadra non può uscire finché ci sono altri membri"}`